

INF01151 – Sistemas Operacionais II

Aula 10 - Comunicação inter-processos por troca de mensagens

Prof. Alberto Egon Schaeffer Filho

Introdução

- Em ambientes de **memória distribuída**, comunicação entre processos é feita por **troca de mensagem**
 - Processos **não compartilham** o mesmo espaço de endereçamento
 - Troca de mensagem
 - **TAMBÉM** pode ser empregada em ambientes de memória compartilhada
 - Capacidades de comunicação e/ou sincronização **dependem das primitivas** fornecidas pela biblioteca
 - `send(message)`
 - `receive(message)`
- Cada uma pode ser implementada com diferentes semânticas!!
- Associado a bibliotecas de comunicação
- Comunicação pode ser
 - Ponto a ponto
 - Grupo (multicast)

Características desejáveis

① Simplicidade e clareza de uso

- Semântica intuitiva

② Uniformidade

- Processos locais e remotos usam a mesma interface de comunicação

③ Eficiência

- Baixo custo de processamento
- Uso otimizado de recursos do sistema

④ Confiabilidade

- Garantir a entrega das mensagens em presença de falhas
- Eliminar (tratar) replicação de mensagens
- Entrega ordenada



Características desejáveis (continuação)

⑤ Correção (associado a comunicação em grupo)

- Atomicidade: todos ou nenhum recebem
- Entrega ordenada: mensagens entregues de acordo com algum critério

⑥ Flexibilidade

- Nem todas as aplicações tem as mesmas necessidades

⑦ Segurança

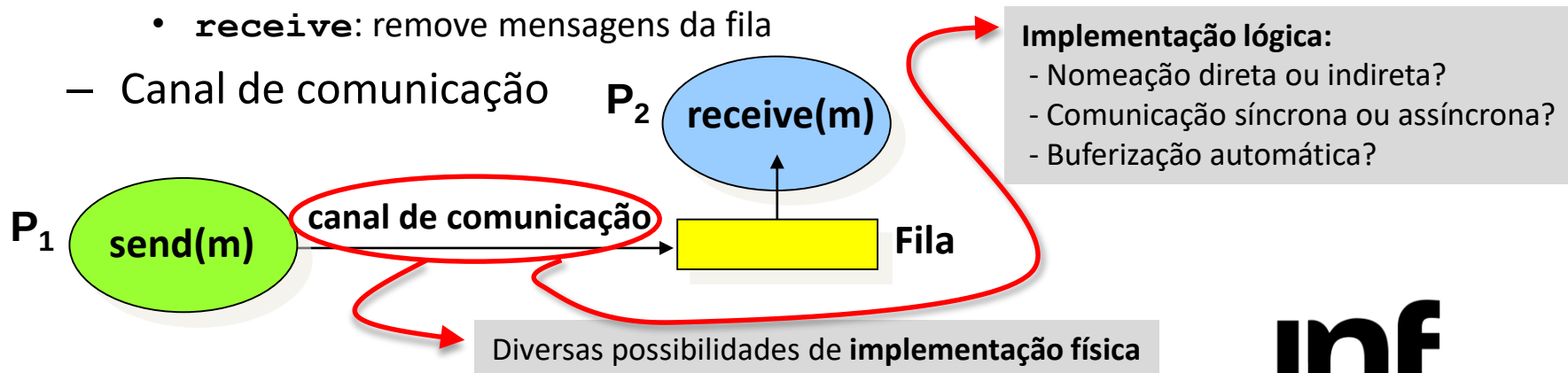
- Autenticação dos pares
- Cifragem das mensagens

⑧ Portabilidade

- Aplicações e da própria biblioteca (heterogeneidade)

Modelo de comunicação

- Duas primitivas básicas: **send** e **receive**
- Modelo
 - Origem (emissor, remetente) e Destino (receptor, destinatário)
 - Mensagem
 - Tamanho **fixo**: implementação do sistema simples, uso é mais difícil
 - Tamanho **variável**: implementação do sistema é mais difícil, uso se torna simples
 - Fila associada ao canal de comunicação
 - **send**: faz com que mensagens sejam adicionadas na fila
 - **receive**: remove mensagens da fila
 - Canal de comunicação



Semânticas das primitivas *send* e *receive*

- Tudo depende da semântica da primitiva...
 - **send**
 - Deve ser indicado um destinatário específico ou a mensagem pode ser entregue a qualquer processo?
 - Remetente fica bloqueado até a mensagem ser aceita pelo destinatário?
 - Há bufeização de mensagens?
 - **receive**
 - Deve ser indicado um remetente específico ou se a mensagem pode ser proveniente de qualquer processo?
 - Receptor fica bloqueado se não houver mensagem a ser recebida?
 - Há bufeização de mensagens?
- Remete a questões de nomeação e sincronismo ...

Nomeação direta ou indireta

- Processos precisam referenciar uns aos outros
 - Indicar se a comunicação é **direta/explícita** ou **indireta/implícita**
 - Nomear ou não o processo com o qual se deseja interagir
- Nomeação **explícita (direta)**
 - Determinação de processo destino/remetente específicos
 - `send(P, message)`: Envia mensagem para processo P
 - `receive(Q, message)`: Recebe mensagem de processo Q
 - Propriedades
 - Canal estabelecido automaticamente entre cada par de processos, precisam apenas saber as identidades de cada processo
 - Exatamente um canal associado a exatamente dois processos
 - Simetria de endereçamento: cada processo precisa saber o nome do outro
 - Variante assimétrica
 - `send(P, message)`: Envia mensagem para processo P
 - `receive("any", message)`: Recebe msg de qualquer processo

Limitação?

Nomeação direta ou indireta

- Nomeação **implícita (indireta)**

Ver também: JavaSpaces, Rinda, PyLinda, etc

- Mensagens são enviadas e recebidas através de **caixas postais**
 - Cada caixa postal possui um **id único**
 - Mensagens podem ser **colocadas** e **retiradas**
- Exemplo típico: fila de mensagens (`MessageQueue` no UNIX)
- Processos só podem se comunicar se compartilham uma caixa postal
 - `send(A, message)`: Envia mensagem para caixa postal A
 - `receive(A, message)`: Recebe mensagem da caixa postal A
- Propriedades
 - Canal só é estabelecido entre um par de processos se ambos compartilham a caixa
 - Canal pode estar associado a mais de dois processos
 - Entre cada par de processos, podem haver diferentes canais. Como?
- P1, P2 e P3 compartilham caixa postal A: **Quem recebe? P2 ou P3?**

Diferentes caixas postais!

```
P1: send(A, message);
```

```
P2: receive(A, message);
```

```
P3: receive(A, message);
```


Comunicação síncrona ou assíncrona

- Comunicação entre processos através das primitivas **send/receive**. Como implementar: bloqueantes ou não?
 - Envio **bloqueante**: processo que faz **send** fica bloqueado até mensagem ser recebida pelo outro processo ou pela mailbox
 - Envio **não-bloqueante**: processo que faz **send** continua a execução após chamar a primitiva
 - Recebimento **bloqueante**: processo que faz **receive** bloqueia até que alguma mensagem esteja disponível
 - Recebimento **não-bloqueante**: processo que faz **receive** obtém uma mensagem válida ou NULL
- Diferentes combinações são possíveis ...
 - Com **send** e **receive bloqueantes**  comunicação síncrona
 - Situação chamada de “rendezvous”
 - Com **send** ou **receive não-bloqueantes**
 - Há apenas mecanismo de comunicação, não de sincronização

Buferização

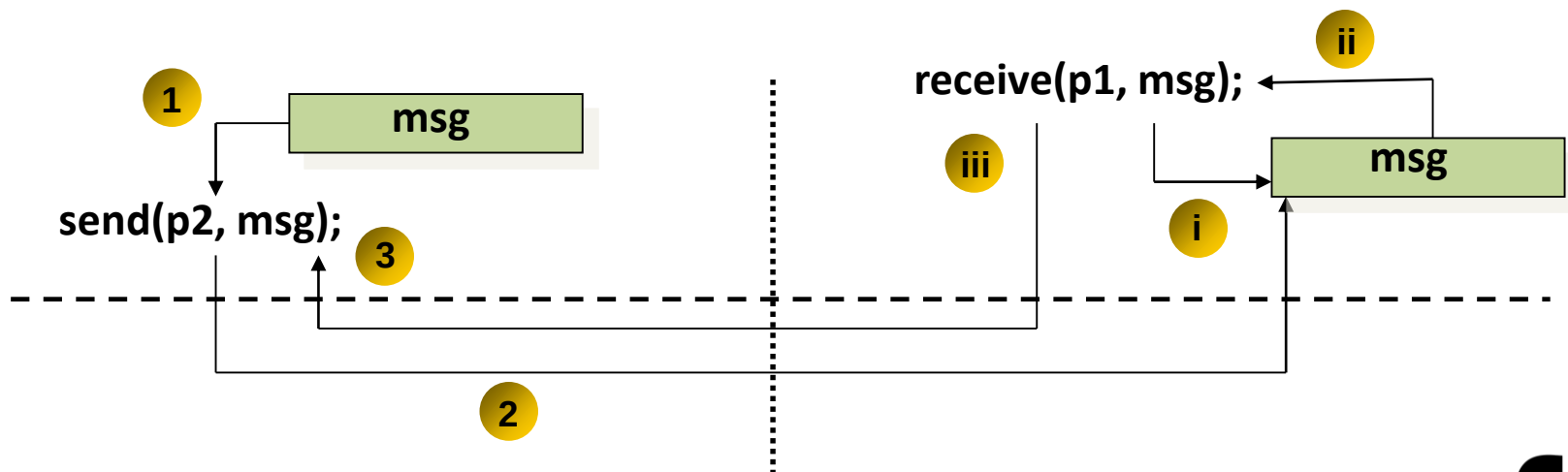
- Em canal de comunicação, direto ou indireto, mensagens são mantidas em fila temporária associada ao canal.
- Três alternativas:
 - **Zero capacity**
 - Fila tem tamanho zero, e canal não pode manter nenhuma mensagem pendente
 - **send** deve ser **bloqueante** até que outro processo possa receber a mensagem
 - Maior eficiência: não é feito cópia local dos dados
 - **Bounded capacity**
 - Fila tem tamanho fixo n , e canal pode manter até n mensagens pendentes
 - Se a **fila não estiver cheia**, quando nova mensagem é enviada **send** é **não-bloqueante**
 - Se a **fila estiver cheia**, quando nova mensagem é enviada **send** deve **bloquear**
 - **Unbounded capacity**
 - **send** sempre é **não-bloqueante**
 - Na prática, não existe; n pode ser muito superior aos casos usuais

Sem buferização

Buferização automática

Implementação comunicação síncrona

- Primitiva **send** e **receive** forem **bloqueantes**
 - Dispensa o uso de fila e buffers
- Procedimento:
 - **send**: processo fica bloqueado até que a recepção seja efetivada
 - **receive**: processo fica bloqueado até que o envio seja feito e a mensagem seja recebida



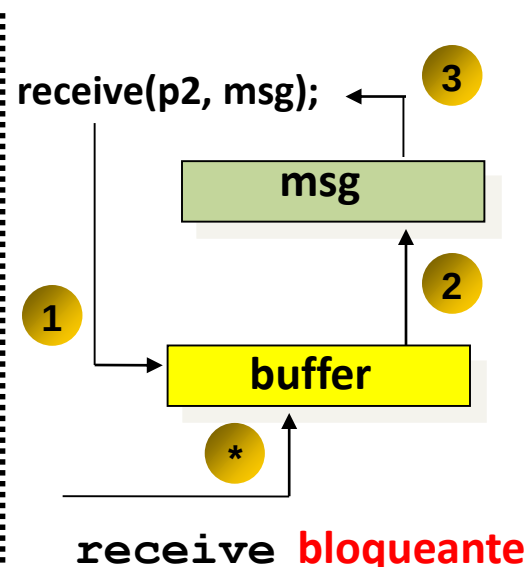
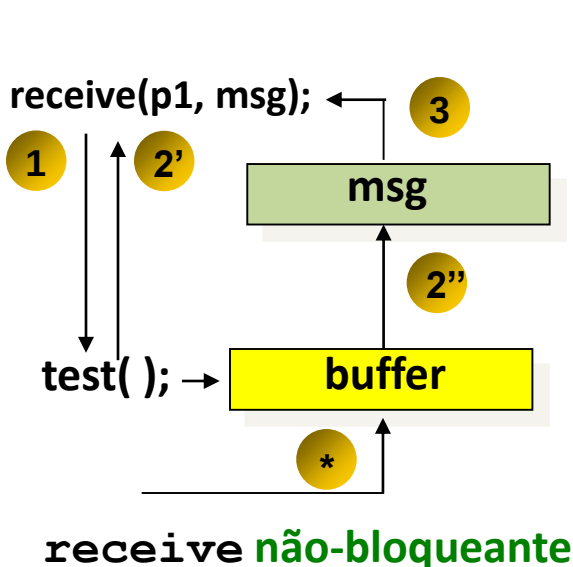
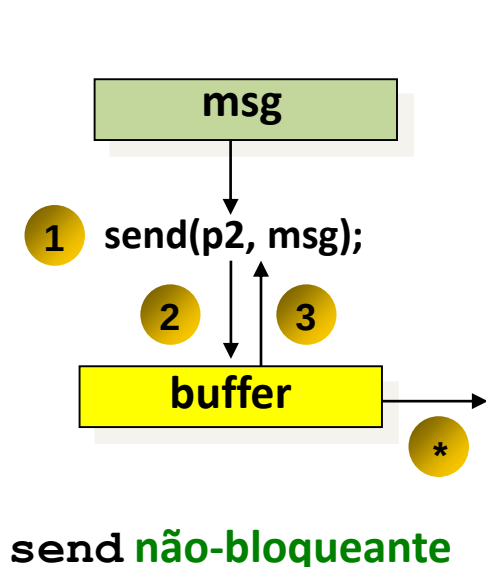
Implementação comunicação assíncrona

- Envolve a utilização de buffers

- **send** é **não-bloqueante**
- **receive** pode ser
 - **Não-bloqueante**
 - **Bloqueante**

E na prática?

Receive é bloqueante: Uma thread pode ficar bloqueada em um receive, enquanto outras threads executam



* Momento de tempo não preciso!

Comparação: comunicação síncrona vs. assíncrona

- Comunicação:
 - **Síncrona:** processos usam versão bloqueante das primitivas
 - **Assíncrona:** processos usam versão não-bloqueante de pelo menos uma das primitivas
- Desvantagens da comunicação síncrona?
 - Limitação da concorrência
 - Possibilidade de *deadlocks*
 - Como??
- Vantagens da comunicação síncrona?
 - Programação simples

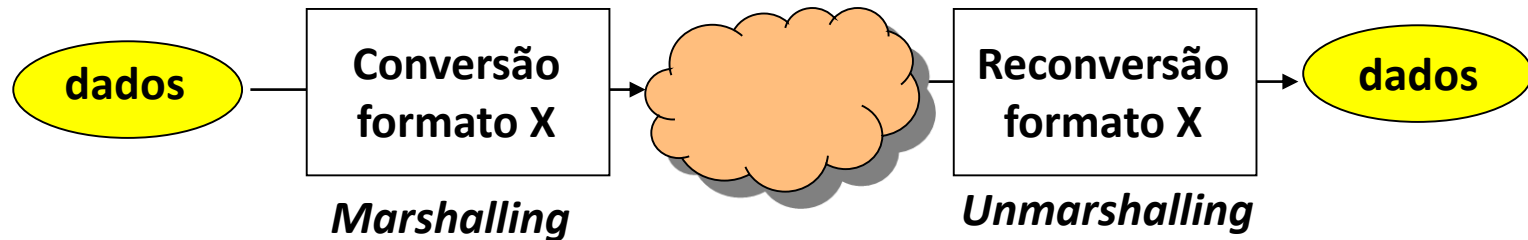
Vantagens e desvantagens da assíncrona são “*dual*” da síncrona

Representação externa de dados

- Informação (dados) é representada em uma estrutura de dados
 - Na comunicação informação é igual a uma seqüência de bytes
 - Necessário converter estrutura de dados $\leftrightarrow$ seqüência de bytes
- Problema se processos estão executando em sistemas remotos
 - Diferentes sistemas = diferentes representações de dados
 - Código ASCII usa 1 byte por caracter vs. Unicode usa 2 bytes por caracter
 - Big endian (byte mais significativo 1º) vs. little endian (byte menos significativo 1º)
 - Formatos de números em ponto flutuante, etc
- Como resolver?
 - ① Representação em um formato independente de máquina
 - Formato de dados acordado (*external data representation*)
 - ② Enviar no formato do emissor e incluir informação sobre o formato empregado

Representação externa de dados

- Marshalling e unmarshalling
 - Conversão de dados em um formato próprio para transmissão
 - Envio: conversão formato local → *external data representation*
 - Recebimento: conversão *external data representation* → formato local
 - Auxilia a tratar a heterogeneidade de representação de dados entre os processos



- Ideal é que seja feito sem envolvimento explícito da aplicação
 - Responsabilidade da biblioteca ou *middleware*
 - Exemplos:
 - **CORBA CDR**: representação de tipos tipos estruturados e primitivos (binário)
 - **Serialização Java**: utilizado para converter objetos em bytestream (binário)
 - **XML/JSON**: utilizado para representar dados trocados por web services

Modelo de falhas

- Processos e/ou canal de comunicação podem falhar
 - Especialmente se **comunicação for distribuída**
 - Transmissões podem falhar, ou serem executadas mais de uma vez
 - Erros de transmissão na rede
- Modelo de falhas
 - Define **como falhas podem se manifestar** de forma a proporcionar um entendimento sobre seus efeitos e conseqüências
- Tipos de falhas:
 - **Omissão**: processo e/ou comunicação não executam sua função
 - Ex: processo termina inesperadamente, mensagem não chega ao destino
 - **Arbitrária (bizantina)**: processo e/ou comunicação executam com erros
 - Ex: mensagem chega no destino com erro, duplicada e/ou fora de ordem
 - **Temporização**: quando limites de tempo não são respeitados
 - Ex: transmissão da mensagem ultrapassa limite estabelecido

Modelo de falhas (continuação)

- Mascaramento de falhas
 - Sistemas geralmente construídos através de coleção de outros componentes
 - Possível construir um sistema confiável a partir de componentes propensos a falhas
 - Quando um sistema **esconde que houve uma falha** ou **transforma falha em um tipo menos grave** (mais “aceitável”)
 - Ex: checksum usado para mascarar mensagem corrompida (bizantina → omissão)
 - Em alguns sistemas, é possível esconder falhas de omissão através de retransmissão
 - Outras técnicas: retransmissão de mensagens, replicação dados/serviços, restaurar processo com info gravada em disco, etc etc
- Confiabilidade na comunicação
 - Definido em termos de:
 - **Validade**: mensagem é em algum momento entregue (garantia)
 - **Integridade**: o que é recebido é exatamente o que foi enviado
 - Tratar duplicação, mensagens espúrias, modificadas

Leituras adicionais

- Silberschatz, A. and Galvin, P. – *“Operating Systems Concepts”*. Wiley, 8th edition, 2009.
 - Capítulo 3 (seções 3.4 e 3.5)
- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G.– *“Distributed Systems: Concepts and Design”* (5th edition), Addison Wesley, 2012
 - Capítulos 2 (seção 2.4), 4 (seções 4.1, 4.2 e 4.3)

Exercícios

1. Implementar o problema do produtor consumidor, utilizando 1 único produtor, 1 único consumidor. Os processos devem produzir/consumir 1 item por vez: produtor não deve produzir duas vezes em sequência, consumidor não deve consumir duas vezes em sequência. Utilize **send/receive**. Faça o esboço da solução.
 - Preciso de buferização?
 - Comunicação direta ou indireta?
 - Send/receive bloqueante ou não-bloqueante?
 - Preciso de primitivas de sincronização como mutex/semáforos?